SCHEDULER ARCADE

A 16-Bit Simulation Engine and Educational Game for Operating System CPU Scheduling Algorithms

Course / Purpose: B.Tech Computer Science and Engineering Capstone Project
Author / Team: Harshil Thakkar (Lead Systems and Frontend Engineer)
Affiliation: Department of Computer Science and Engineering
Repository: https://github.com/harshil5506/scheduler-arcade
Date: October 2026

Abstract: Operating system CPU scheduling concepts are difficult to teach through static Gantt charts. Students memorize formulas for average waiting time and turnaround time, yet they struggle to develop intuition for dynamic trade-offs such as the convoy effect, quantum tuning overhead, and starvation. We designed and built Scheduler Arcade, an interactive browser-based simulator disguised as a 16-bit retro arcade game. The engine simulates First-Come First-Served (FCFS), Shortest Job First (SJF/SRTF), Priority Scheduling with Aging, Round Robin (RR), and Multi-Level Feedback Queue (MLFQ). The project relies on plain ES Modules, HTML5 Canvas, and Web Audio API without frameworks, build tools, or external assets. Verification is enforced through an automated unit test suite of 25 textbook benchmarks. A live kernel decision terminal and interactive diagnostic overlays make scheduling decisions explainable tick by tick.

Table of Contents

• 1. Problem Statement
• 2. Project Objectives
• 3. Existing Tools and Gaps
• 4. System Design
• 5. Implementation Details
• 6. Testing and Benchmark Verification
• 7. Engineering Decisions and Trade-offs
• 8. Challenges and Solutions
• 9. Team Contributions and Milestone Log
• 10. Limitations and Future Work
• 11. References
• Appendix: Textbook Verification Benchmarks

1. Problem Statement

In undergraduate computer science curricula, CPU scheduling is traditionally taught through textbook problem sets (Silberschatz et al.). Students calculate waiting times by hand on paper Gantt charts for static sets of four or five jobs.

This pedagogical method fails in three specific ways:

- **Offline assumption:** It treats process arrival as an offline problem where all future arrivals and bursts are already known, hiding the runtime reality that operating systems make decisions online with incomplete data.
- **Zero-cost abstraction:** It treats context switching as a zero-cost abstraction, obscuring how small time slices in Round Robin degrade throughput through scheduling overhead.

- **Opaque failure states:** Students cannot see the cause of runtime anomalies such as starvation and priority inversion in real time. They see the final numerical penalty, but not the tick by tick kernel decisions that caused it.

2. Project Objectives

We set out to build a simulator with four concrete targets:

- Implement five classic scheduling algorithms in a mathematically rigorous simulation engine matching textbook definitions.
- Render the process queue, active CPU execution, and timeline synchronously at 60 frames per second using procedural pixel graphics and procedural 8-bit sound synthesis, with zero external graphic or audio downloads.
- Provide interactive explainability tools, specifically a real-time kernel decision trace terminal and an inspection modal explaining why each task is running or waiting.
- Verify all algorithmic implementations against automated unit tests covering standard textbook benchmarks, edge cases, and arrival gaps.

3. Existing Tools and Gaps

We reviewed existing educational schedulers commonly used in academic coursework and viva preparation:

Tool	Core Technology	Primary Advantage	Identified Gap
Java CPU Simulators (CPUSim)	Java Swing / AWT	Detailed parameter inputs	Requires JRE installation; outdated UI; static execution graphs.
Web Gantt Visualizers	React / Vue libraries	Simple web accessibility	Static post-facto calculation; lacks preemption animation, context switch cost, and gamification.
CLI Python Scripts	Python terminal	Algorithmic tracing	No visual intuition for queue buildup, convoy effect, or live starvation rates.
Scheduler Arcade (Our Work)	Vanilla JS / Canvas 2D	Zero install; live step debugger; real-time audio; algorithmic overlays	Focused strictly on uniprocessor scheduling; multi-core SMP is not modeled.

Table 1: Comparison of existing educational scheduling tools with Scheduler Arcade.

4. System Design

The application runs entirely on the client side in the browser. It is organized into modular JavaScript components with strict separation between simulation state, rendering, and sound synthesis.

[INSERT: Figure 1 - Architecture Block Diagram showing data flow between SimulationEngine, PixelRenderer, SoundEngine, and DOM Overlay Controller]

Figure 1: Component architecture and runtime data flow.

- **SimulationEngine (engine.js, scheduler.js):** Pure state machine maintaining simulation clock tick, ready queue, running job reference, context switch freeze counter, SLA hearts, and historical slice timeline. Emits structured lifecycle events (arrival, dispatch, context_switch, complete, preempt, sla_violation, kernel_log).
- **PixelRenderer (ui.js):** Canvas 2D engine operating at 1000x620 resolution with imageSmoothingEnabled = false. Procedurally generates sprite sheets on offscreen canvases, renders parallax circuit background, timeline Gantt strip, physical CPU socket, starvation meters, and decision badges.

- **SoundEngine (audio.js):** Web Audio API synthesizer constructing square, triangle, and noise waves dynamically to signal dispatches, preemption, context switches, and failure alerts without external audio files.
- **GameController (main.js):** Glues keyboard and mouse input, tick step loops, overlay modal navigation, and hit-testing for interactive process inspection.

5. Implementation Details

5.1 Pure Algorithm Logic (scheduler.js)

We implemented pure deterministic scheduling functions that serve both as simulation drivers and as automated unit test fixtures:

- **FCFS:** Strict queue order based on arrival tick. Non-preemptive.
- **SJF / SRTF:** Selection based on remaining burst time. Preemption triggers if an incoming job has remaining burst strictly smaller than current running job.
- **Priority with Dynamic Aging:** Priority numbers follow UNIX standards where lower integers indicate higher priority. To resolve starvation, an optional aging parameter decrements waiting tasks' effective priority by 1 after every k waiting ticks.
- **Round Robin:** Processes receive a time quantum q. If remaining burst exceeds q, the task yields the CPU, pays the configured context switch cost, and appends to the queue tail after collecting any tasks arriving on that exact tick.
- **MLFQ:** Three priority tiers (Q0 with q=2, Q1 with q=4, and Q2 running FCFS). CPU-heavy tasks demote down tiers upon slice exhaustion. An optional periodic boost resets all tasks back to Q0.

5.2 Educational Explainability Modules

Following initial testing, we noticed that while the platformer metaphor was visually lively, users could not understand why certain jobs were chosen. We implemented three explainability features:

1. **Physical CPU Socket:** A defined hardware socket on canvas ([CPU CORE]) showing whether the core is computing, paying context switch overhead, or idle. For Round Robin and MLFQ, a countdown bar displays SLICE: q_rem/q above the socket.
2. **Algorithm Decision Badges:** FCFS displays queue ranks (#1 NEXT, #2). SJF highlights the shortest burst job with a pulsating [★ MIN] badge. Priority crowns the highest priority task with [■ VIP] and flags promotions with [▲ AGED]. MLFQ marks jobs with tier labels [Q0], [Q1], or [Q2].
3. **Live Kernel Decision Log:** A monospace CRT terminal docked in the bottom-right corner. It prints human-readable rationales on every tick, such as: TICK 2: DISPATCH -> P2 allocated CPU. Reason: [SJF] Shortest burst in queue (2 ticks remaining).
4. **Interactive Process Inspector:** Clicking any process on screen pauses execution and opens a modal displaying PID, state, arrival, burst, remaining ticks, priority, waiting time, response time, and an academic explanation of its current condition.

[INSERT: Figure 2 - Screenshot of Canvas showing the illuminated CPU Core Socket, Ready Queue Badges, and the docked Kernel Decision Log Terminal]

Figure 2: Real-time simulation view with decision overlays and kernel trace pane.

6. Testing and Benchmark Verification

We wrote an automated test suite executed via node tests.js. It runs 25 benchmark cases without browser dependencies, comparing calculated metrics against standard theoretical values from Silberschatz et al.

Test Group	Evaluated Algorithm	Workload Characteristics	Expected WT	Measured WT	Result

Test 1	FCFS	P1(24), P2(3), P3(3) arriving at 0	17.00 ticks	17.00 ticks	**PASS**
Test 2	SJF Non-preemptive	P1(6,arr 0), P2(8,arr 1), P3(7,arr 2), P4(3,arr 3)	7.00 ticks	7.00 ticks	**PASS**
Test 3	SRTF Preemptive	Same workload as Test 2 with preemption	6.50 ticks	6.50 ticks	**PASS**
Test 4	Priority Non-preemptive	P1(10,p3), P2(1,p1), P3(2,p4), P4(1,p5), P5(5,p2)	8.20 ticks	8.20 ticks	**PASS**
Test 5	Round Robin (q=4)	P1(24), P2(3), P3(3) arriving at 0	5.67 ticks	5.67 ticks	**PASS**
Test 6	Priority with Aging	Low priority job waiting under higher burst stream	Clear in 12 ticks	Clear in 12 ticks	**PASS**
Test 7	MLFQ	Mixed CPU-heavy and interactive I/O tasks	Wait <= 2 ticks	Wait = 1 tick	**PASS**
Test 8	Edge Cases	Empty pool, single task, arrival gaps	Gaps detected	Gaps detected	**PASS**

Table 2: Automated unit test verification summary across 25 textbook test cases.

[INSERT: Figure 3 - Terminal output screenshot showing 25/25 automated unit test passes in tests.js]

Figure 3: Output of the test suite verifying mathematical correctness.

7. Engineering Decisions and Trade-offs

1. **Vanilla JavaScript vs React / Svelte:** Chosen plain ES Modules and direct DOM manipulation. Rejected React and Svelte. Trade-off: Manual modal toggles and UI updates were required, but eliminated all bundlers, node_modules build friction, and external dependencies.
2. **Procedural Pixel Art vs Pre-rendered PNG Sprites:** Chosen offscreen canvas pixel drawing. Rejected external PNG sprite files. Trade-off: Writing coordinate grid arrays was tedious, but reduced image asset download size to zero and guarantees crisp rendering with imageSmoothingEnabled = false.
3. **Web Audio API Synthesis vs Audio Files:** Chosen procedural oscillator synthesis (square, triangle, noise). Rejected MP3/WAV assets. Trade-off: Audio sounds strictly like vintage 8-bit arcade hardware, but initial page weight dropped by several megabytes, avoiding asset loading latency.
4. **Synchronous Step Engine vs Timestamp-Driven Simulation:** Chosen integer tick stepping coupled with an accumulator loop. Rejected continuous millisecond deltas. Trade-off: Sub-tick animations require frame interpolation, but scheduling decisions remain 100% deterministic and reproducible across machines.
5. **Local Storage vs Remote Backend:** Chosen browser localStorage for stars and configuration. Rejected remote database. Trade-off: Scores do not sync across different user machines, but the application requires zero authentication, stores no telemetry, and operates completely offline.

8. Challenges and Solutions

Challenge 1: Visual Metaphor Inversion

• *Symptom:* During early review, users familiar with operating systems stated that the game was hard to follow despite knowing the algorithms.

• *Cause:* The character was walking across the ground while processes trailed behind it. There was no distinct representation of the CPU socket, so users could not tell which process was executing versus which was waiting.

• *Solution:* We anchored an illuminated CPU socket ([CPU CORE]) on the canvas, added flow arrows from ready queue into the core, placed a quantum countdown above the socket, and added the real-time Kernel Decision Log terminal.

Challenge 2: Ready Queue Enqueueing During Round Robin Time Slice Expiration
• *Symptom:* In Round Robin testing, tasks arriving on tick T were occasionally placed behind the task whose quantum expired on tick T, violating the rule that newly arrived tasks enter the ready queue before preempted tasks re-enter.
• *Cause:* The simulation engine advanced time and checked arrivals after re-appending the expired job to the queue tail.
• *Solution:* We restructured stepTick() in engine.js so that when a quantum expires, the preempted task is held in a temporary variable, time advances, new arrivals are collected into the ready queue, and only then is the preempted task appended to the tail.

Challenge 3: Inaudible Audio on First Page Load
• *Symptom:* Audio effects failed to play when users clicked start controls.
• *Cause:* Modern browsers enforce autoplay security policies that suspend AudioContext until an explicit user interaction event occurs.
• *Solution:* We wrapped audio.resume() inside a one-time global click listener on the canvas and window, automatically waking the audio subsystem on the user's first interaction.

9. Team Contributions and Milestone Log

Member	Responsibilities	Key Deliverables
Harshil Thakkar	Simulation logic, canvas renderer, explainability tools, test suite	scheduler.js, engine.js, ui.js, audio.js, tests.js, GitHub deployment

Milestone	Target Date	Status	Verification
M1: Algorithm Engine & Unit Tests	**[INSERT: Date 1]**	Completed	25/25 tests passing in tests.js
M2: Procedural Canvas Renderer & HUD	**[INSERT: Date 2]**	Completed	60 FPS pixel rendering on Canvas 2D
M3: Sound Engine & Campaign Puzzles	**[INSERT: Date 3]**	Completed	Web Audio synthesizer; 8 puzzle missions
M4: Advanced Modes (Versus, Draft, Boss)	**[INSERT: Date 4]**	Completed	Split-screen dual engine; boss mechanics
M5: Educational Explainability Upgrade	**[INSERT: Date 5]**	Completed	Kernel log terminal; diagnostic modal
M6: GitHub Release & Documentation	**[INSERT: Date 6]**	Completed	Public repository pushed to origin/main

10. Limitations and Future Work

To maintain engineering credibility, we document the following known limitations:

- **Single-Core Architecture Only:** The engine models a uniprocessor system. It does not simulate symmetric multiprocessing (SMP), multi-core processor affinity, or NUMA memory node latency.
- **Simplified Process Model:** Jobs are defined strictly by arrival tick, CPU burst length, static priority, and process type. The engine does not model dynamic multi-burst behavior where a process alternates between CPU bursts and blocking I/O waits.
- **Canvas Hit-Testing Sensitivity:** Process card hit-testing relies on screen coordinate bounding boxes calculated during rendering. In rapid screen shake animations, clicking a small card can occasionally miss on the first attempt.
- **Mobile Touch Support:** While the interface layout scales to viewport dimensions, control buttons and keyboard shortcuts are optimized for physical keyboards and desktop mouse inputs. Mobile touch drag gestures are not currently

implemented.

Future Work: Implement multi-core scheduling with independent per-core run queues and load-balancing work stealing. Add interleaved CPU and I/O burst sequences with a visible blocked wait queue. Introduce an interactive Gantt chart drag-and-drop mode where students manually construct the optimal schedule against the automated solver.

11. References

1. Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*, 10th Edition. John Wiley & Sons, 2018. Chapter 5 (CPU Scheduling).
2. Tanenbaum, A. S., and Bos, H. *Modern Operating Systems*, 4th Edition. Pearson, 2014. Chapter 2 (Processes and Threads).
3. MDN Web Docs. *Canvas API* and *Web Audio API Documentation*. Mozilla Foundation, 2024.

Appendix: Textbook Verification Benchmarks

```
// Textbook Benchmark: FCFS vs SJF vs Round Robin (Silberschatz et al.)
const textbookWorkload = [
  { pid: 'P1', arrival: 0, burst: 24, priority: 3, type: 'cpu' },
  { pid: 'P2', arrival: 0, burst: 3,  priority: 1, type: 'io' },
  { pid: 'P3', arrival: 0, burst: 3,  priority: 2, type: 'cpu' }
];

// FCFS Execution: P1 (0..24), P2 (24..27), P3 (27..30)
// Waiting times: P1=0, P2=24, P3=27. Average WT = 17.00 ticks.

// Round Robin (q=4) Execution: P1(0..4), P2(4..7), P3(7..10), P1(10..30)
// Waiting times: P1=6, P2=4, P3=7. Average WT = 5.67 ticks.
```

TO FILL IN (Submission Checklist)

- Figure 1: Insert Architecture Block Diagram export.
- Figure 2: Insert browser screenshot showing CPU Socket, Badges, and Terminal Log.
- Figure 3: Insert terminal output screenshot of running 'node tests.js'.
- Table 4: Replace [INSERT: Date 1] through [INSERT: Date 6] with your course semester dates.